

Manual do Programa DIS

Estrutura de Funções em Deep Inelastic Scattering no Formalismo de Dipolos

Prof. Dr. Rafael Camargo Rodrigues de Lima
Universidade do Estado de Santa Catarina (UDESC)
Colaboração DUNE

Sumário

1	Introdução	2
2	Como Utilizar o Programa	3
2.1	Estrutura Geral	3
2.2	Compilação	3
2.3	Executando os Programas	3
2.4	Arquivos de Saída	4
2.5	Visualização dos Resultados	4
2.6	Parâmetros Físicos Importantes	4
2.7	Relações Cinemáticas Fundamentais	5
2.8	Função de Estrutura Total	5
2.9	Seção de Choque Diferencial	5
3	Como o Programa Funciona	5
3.1	Formalismo de Dipolos	5
3.2	Funções de Onda	6
3.3	Funções de Estrutura	6
3.4	Modelo GBW	6
3.5	Modelo IIM	7
3.6	Função $x F_3$	7
3.7	Fluxo Numérico do Código	7
3.8	Visualização Física	7
4	Considerações Finais	8
4.1	Utilização do Makefile	8
4.1.1	Compilação Principal	8
4.1.2	Limpeza dos Arquivos Compilados	9
4.1.3	Execução do Scan em x	9
4.1.4	Cálculo da Seção de Choque	9
4.1.5	Execução dos Scripts Python	10
4.1.6	Fluxo Completo Automatizado	10
4.1.7	Alteração de Parâmetros	11

1 Introdução

O programa DIS foi desenvolvido para calcular funções de estrutura em espalhamento profundamente inelástico (Deep Inelastic Scattering — DIS) utilizando o formalismo de dipolos da QCD em pequenos valores da variável de Bjorken x .

O código implementa:

- funções de onda eletrofracas do bóson intermediário;
- modelos fenomenológicos para a seção de choque dipolo-próton;
- cálculo das funções de estrutura:

$$F_T, \quad F_L, \quad F_2, \quad xF_3;$$

- comparação entre modelos fenomenológicos:
 - GBW;
 - IIM.

O objetivo principal do programa é estudar:

- QCD em pequenos x ;
- saturação gluônica;
- espalhamento DIS;
- neutrinos ultraenergéticos.

2 Como Utilizar o Programa

2.1 Estrutura Geral

O código é dividido em módulos físicos independentes.

Os principais arquivos são:

- `wavefunctions.cpp`
- `dipole_models.cpp`
- `weak_structure_functions.cpp`
- `sigma_nuN.cpp`
- `scan_x.cpp`

Os scripts Python de visualização são:

- `plot_scan_x.py`
- `plot_scan_x_models.py`
- `plot_wavefunctions.py`

2.2 Compilação

A compilação é realizada via:

```
make
```

ou:

```
make all
```

Os executáveis são colocados normalmente em:

```
build/
```

Para limpar arquivos compilados:

```
make clean
```

2.3 Executando os Programas

Exemplo de execução:

```
./build/scan_x
```

Esse programa calcula as funções de estrutura em função de x , mantendo Q^2 fixo.

Outro exemplo:

```
./build/sigma_nuN
```

que calcula a seção de choque neutrino-nucleon.

2.4 Arquivos de Saída

Os resultados são salvos em arquivos ASCII simples.

Exemplo:

`data/scan_x_F2_CC.dat`

com colunas:

$$x, \quad Q^2, \quad F_T, \quad F_L, \quad F_2.$$

Outro exemplo:

`data/scan_x_F2_CC_models.dat`

contendo comparação entre os modelos GBW e IIM.

2.5 Visualização dos Resultados

Os gráficos podem ser gerados com:

`python scripts/plot_scan_x.py`

ou:

`python scripts/plot_scan_x_models.py`

e:

`python scripts/plot_wavefunctions.py`

Os gráficos são salvos em:

`plots/`

2.6 Parâmetros Físicos Importantes

Os cálculos dependem principalmente de:

- energia do neutrino:

$$E_\nu;$$

- virtualidade:

$$Q^2;$$

- variável de Bjorken:

$$x;$$

- massa do bóson intermediário:

$$M_W, \quad M_Z;$$

- modelo fenomenológico do dipolo.

2.7 Relações Cinemáticas Fundamentais

A variável de Bjorken é dada por:

$$x = \frac{Q^2}{2m_N\nu} \quad (1)$$

onde:

- m_N é a massa do nucleon;
- ν é a energia transferida.

A massa invariante hadrônica é:

$$W^2 = Q^2 \left(\frac{1}{x} - 1 \right) + m_N^2 \quad (2)$$

2.8 Função de Estrutura Total

O programa calcula:

$$F_2(x, Q^2) = F_T(x, Q^2) + F_L(x, Q^2) \quad (3)$$

onde:

- F_T representa a componente transversal;
- F_L representa a componente longitudinal.

2.9 Seção de Choque Diferencial

A seção de choque diferencial para DIS neutrino-nucleon é:

$$\frac{d^2\sigma^{\nu N}}{dx dQ^2} = \frac{G_F^2}{2\pi x} \left(\frac{M_W^2}{Q^2 + M_W^2} \right)^2 [Y_+ F_2 - y^2 F_L + Y_- x F_3] \quad (4)$$

onde:

$$Y_{\pm} = 1 \pm (1 - y)^2 \quad (5)$$

e:

$$y = \frac{Q^2}{xs} \quad (6)$$

3 Como o Programa Funciona

3.1 Formalismo de Dipolos

No regime de pequenos x , o bóson intermediário flutua em um dipolo quark-antiquark antes da interação hadrônica:

$$W^{\pm}/Z^0 \rightarrow q\bar{q} \quad (7)$$

A interação do dipolo com o próton é descrita pela seção de choque dipolar:

$$\sigma_{dip}(x, r) \quad (8)$$

onde:

- r é o tamanho transversal do dipolo;
- x controla a densidade gluônica.

3.2 Funções de Onda

As funções de onda descrevem a probabilidade de o bóson produzir um dipolo.

O programa calcula:

$$|\Psi_T(r, z, Q^2)|^2 \quad (9)$$

e:

$$|\Psi_L(r, z, Q^2)|^2 \quad (10)$$

onde:

- z é a fração de momento longitudinal do quark;
- r é a separação transversal do dipolo.

3.3 Funções de Estrutura

As funções de estrutura são calculadas integrando sobre:

- tamanho do dipolo r ;
- fração longitudinal z .

A componente transversal é:

$$F_T(x, Q^2) = \frac{Q^2}{4\pi^2} \int d^2r \int_0^1 dz |\Psi_T|^2 \sigma_{dip}(x, r) \quad (11)$$

A componente longitudinal é:

$$F_L(x, Q^2) = \frac{Q^2}{4\pi^2} \int d^2r \int_0^1 dz |\Psi_L|^2 \sigma_{dip}(x, r) \quad (12)$$

3.4 Modelo GBW

O modelo GBW implementa saturação gluônica através da parametrização:

$$\sigma_{dip}(x, r) = \sigma_0 \left[1 - \exp \left(-\frac{r^2 Q_s^2(x)}{4} \right) \right] \quad (13)$$

onde a escala de saturação é:

$$Q_s^2(x) = \left(\frac{x_0}{x} \right)^\lambda \quad (14)$$

3.5 Modelo IIM

O modelo IIM melhora a transição entre:

- regime diluído;
- regime saturado.

O modelo incorpora:

- dinâmica BFKL;
- efeitos de saturação;
- matching contínuo entre regimes físicos.

3.6 Função xF_3

A função:

$$xF_3 \tag{15}$$

é sensível à diferença entre distribuições de quarks e antiquarks.

Ela aparece naturalmente devido à estrutura vetorial-axial da interação fraca.

3.7 Fluxo Numérico do Código

O programa executa os seguintes passos:

1. definição de:

$$x, \quad Q^2;$$

2. cálculo das funções de onda:

$$|\Psi_T|^2, \quad |\Psi_L|^2;$$

3. cálculo da seção de choque dipolar:

$$\sigma_{dip}(x, r);$$

4. integração numérica em:

$$r, \quad z;$$

5. construção das funções:

$$F_T, \quad F_L, \quad F_2, \quad xF_3;$$

6. cálculo da seção de choque:

$$\sigma^{\nu N}.$$

3.8 Visualização Física

Os scripts Python permitem:

- scans em x ;
- comparação entre modelos;
- mapas de calor;
- visualização das funções de onda.

As escalas logarítmicas são importantes devido às múltiplas ordens de grandeza envolvidas.

4 Considerações Finais

O programa DIS fornece uma implementação modular do formalismo de dipolos aplicado ao espalhamento profundamente inelástico de neutrinos.

A estrutura do código permite:

- estudar saturação gluônica;
- analisar funções de estrutura em pequenos x ;
- comparar parametrizações fenomenológicas;
- explorar seções de choque em energias extremas.

O código também pode servir como base para futuras extensões envolvendo:

- PDFs nucleares;
- núcleos pesados;
- evolução DGLAP/BFKL;
- correções de ordem superior;
- transporte de neutrinos ultraenergéticos.

4.1 Utilização do Makefile

O projeto utiliza um **Makefile** para automatizar:

- compilação;
- criação de diretórios;
- execução dos programas;
- geração de gráficos.

Isso permite executar todo o pipeline científico com poucos comandos.

4.1.1 Compilação Principal

Para compilar todos os programas:

```
make
```

ou:

```
make all
```

O Makefile compila automaticamente os arquivos:

- `main.cpp`
- `scanx.cpp`
- `sigmanuN.cpp`
- módulos físicos auxiliares.

Os executáveis são normalmente colocados em:

```
build/
```


4.1.2 Limpeza dos Arquivos Compilados

Para remover objetos compilados e executáveis:

```
make clean
```

Isso normalmente remove:

- arquivos .o;
- executáveis;
- arquivos temporários de build.

4.1.3 Execução do Scan em x

Para calcular as funções de estrutura em função de x :

```
make scan-x
```

ou diretamente:

```
./build/scan_x
```

O programa produz arquivos como:

```
data/scan_x_F2_CC.dat
```

contendo:

$$x, \quad Q^2, \quad F_T, \quad F_L, \quad F_2.$$

4.1.4 Cálculo da Seção de Choque

Para calcular a seção de choque neutrino-nucleon:

```
make sigma
```

ou:

```
./build/sigma_nuN
```

O código integra numericamente:

$$\sigma^{\nu N} = \int dx \int dQ^2 \frac{d^2 \sigma^{\nu N}}{dx dQ^2} \quad (16)$$

usando a seção diferencial:

$$\frac{d^2 \sigma^{\nu N}}{dx dQ^2} = \frac{G_F^2}{2\pi x} \left(\frac{M_W^2}{Q^2 + M_W^2} \right)^2 [Y_+ F_2 - y^2 F_L + Y_- x F_3] \quad (17)$$

4.1.5 Execução dos Scripts Python

Os gráficos podem ser gerados automaticamente via Makefile.

Por exemplo:

```
make plot-scan-x
```

que executa:

```
python scripts/plot_scan_x.py
```

Outro exemplo:

```
make plot-models
```

que executa:

```
python scripts/plot_scan_x_models.py
```

Para os mapas de calor das funções de onda:

```
make plot-wavefunctions
```

executando:

```
python scripts/plot_wavefunctions.py
```

4.1.6 Fluxo Completo Automatizado

O fluxo típico de utilização do programa é:

1. compilar:

```
make
```

2. executar o cálculo:

```
make scan-x
```

3. gerar os gráficos:

```
make plot-scan-x
```

4. visualizar os resultados em:

```
plots/
```

4.1.7 Alteração de Parâmetros

Os parâmetros físicos podem ser modificados diretamente nos arquivos-fonte.

Os mais importantes são:

- energia do neutrino:

$$E_\nu;$$

- virtualidade:

$$Q^2;$$

- modelo dipolar:

$$\text{GBW} \quad \text{ou} \quad \text{IIM};$$

- limites de integração;

- resolução numérica das integrais.

Após qualquer modificação, recomenda-se recompilar:

```
make clean  
make
```